Adaptive Drone Swarm Behavior Simulation

Project Report

Prepared By:

Anup Baigain

Madhusudan Pun

Toran Bahadur Niroula

Submitted To:

Samsung Innovation Campus

Pulchowk Campus

Date:

May 8, 2025

Abstract

This project details the design and implementation of a 3D simulation environment for analyzing and visualizing adaptive drone swarm behaviors. Developed using Python and its libraries `tkinter` for the graphical user interface and `matplotlib` for 3D visualization, the simulation incorporates key bio-inspired algorithms. These include classical Boids behaviors (cohesion, alignment, and separation) for decentralized flocking, and a Hybrid Control System combining PD control for dynamic pattern formation and PID control for robust obstacle avoidance and goal seeking. The platform allows for the simulation of complex scenarios involving static cylindrical obstacles. A key focus was on creating a modular and extensible architecture to facilitate future research, such as integrating reinforcement learning for emergent strategies and exploring role-based task allocation within the swarm. This work serves as a practical application of Python programming skills acquired through the Samsung Innovation Campus, providing a foundation for understanding multi-agent systems and their potential in autonomous aerial robotics.

Contents

Abstract **1**

1 Introduction **3**

2 Project Objectives and Scope **3**
2.1 Core Objectives 3
2.2 Scope and Deliverables 3

3 Software Architecture Overview **4**

4 Implementation Details **4**
4.1 Core Classes 4
4.2 Swarm Behaviors 5
4.3 Advanced Features 8
4.4 User Interaction 10

5 Results and Discussion **11**
5.1 Demonstrated Behaviors 11
5.2 Visualization and Analysis 11
5.3 Human Element and Reflection 11
5.4 Extensibility 12

6 Conclusion **12**

A Source Code **14**

1 Introduction

Drone swarm technology holds significant potential across various domains, from environmental monitoring and search and rescue operations to complex logistics and infrastructure inspection. The ability of multiple autonomous aerial vehicles to coordinate and act as a single intelligent entity offers distinct advantages in terms of efficiency, robustness, and scalability compared to single-agent systems. This project aims to explore the fundamental principles governing such collective behaviors by developing a realistic and interactive 3D simulation environment. Inspired by natural phenomena like bird flocking and fish schooling, the simulation models emergent behaviors through the application of distributed control algorithms. The primary objective is to create a flexible platform in Python that not only demonstrates core swarm intelligence concepts but also serves as a foundation for developing and testing more advanced coordination strategies for adaptive drone swarms in complex and dynamic environments. This report outlines the design, implementation, and evaluation of this simulation platform.

2 Project Objectives and Scope

This project was guided by a set of core objectives aimed at building a functional and extensible drone swarm simulation platform. The scope was defined to ensure achievable deliverables within the project timeline.

2.1 Core Objectives

- **Develop a Modular Simulation Platform:** Design and implement a flexible Python-based simulation environment capable of supporting various swarm behaviors and scenarios.
- **Implement Fundamental Swarm Behaviors:** Incorporate key decentralized control algorithms including cohesion, alignment, and separation, drawing inspiration from the Boids model.
- **Design Adaptive Collision Avoidance:** Implement mechanisms for drones to detect and avoid both static obstacles (like pillars) and dynamic obstacles (other drones) in the 3D environment.
- **Enable Dynamic Pattern Formation:** Allow the swarm to dynamically arrange themselves into predefined geometric patterns (e.g., circle, square, line, star, triangle, '1', 'S') and manage transitions between these formations.
- **Extend to 3D Simulation and Explore Advanced Concepts:** Implement the simulation in a 3D space for a more realistic representation and lay the groundwork for integrating advanced features like reinforcement learning for enhanced adaptability.

2.2 Scope and Deliverables

The project's scope encompassed the following key areas, with defined deliverables to demonstrate successful implementation:

- **Physics-Based Drone Movement and Collision Detection:** Implement realistic drone kinematics, including velocity updates based on calculated forces and basic collision detection with obstacles and other drones.
- **Swarm Intelligence Algorithms:** Successful implementation and demonstration of cohesion, alignment, and separation rules influencing drone movement.

- **Obstacle Avoidance:** Development of algorithms enabling drones to navigate around static pillar obstacles and avoid collisions with other swarm members.
- **User-Selectable Pattern Formations:** Implementation of functions to generate target positions for various patterns and mechanisms for drones to transition to and maintain these formations.
- **Visualization and Interactive GUI:** Creation of a graphical user interface using `tkinter` to control simulation parameters, switch modes, and visualize the swarm's behavior in real-time 3D using `matplotlib`.
- **Technical Documentation and Performance Analysis:** Provision of code documentation and analysis of simulation results, including metrics like success rates and completion times in obstacle avoidance scenarios.

3 Software Architecture Overview

The simulation platform is designed with a modular architecture to ensure flexibility, maintainability, and extensibility. The system is primarily composed of the following key modules:

- **Physics Engine:** This module is responsible for the low-level simulation of drone movement. It calculates the change in position based on velocity and the application of forces. It also includes the logic for detecting potential collisions between drones and with environmental obstacles.
- **Swarm Behavior Module:** This is the core of the swarm intelligence implementation. It houses the algorithms for decentralized control, including the Boids-inspired rules for cohesion, alignment, and separation, which govern the local interactions between drones. It also incorporates the logic for obstacle avoidance and goal-seeking behaviors, often utilizing control strategies like PID.
- **Visualization System:** This module handles the graphical representation of the simulation. It uses `tkinter` to create the interactive user interface, allowing users to control simulation parameters and modes. `matplotlib`'s 3D plotting capabilities are utilized to visualize the drones, obstacles, and simulation boundaries in real-time.
- **Scenario Manager:** This component is responsible for setting up the simulation environment. It defines the boundaries of the simulation space, the initial positions of the drones, the locations and dimensions of static obstacles (pillars), and the target positions for dynamic pattern formations.

4 Implementation Details

This section delves into the specifics of how the core components and behaviors of the drone swarm simulation were implemented in Python.

4.1 Core Classes

The simulation is built around several key Python classes, each encapsulating the properties and behaviors of different elements within the simulation:

- **Pillar:** This class represents a static, cylindrical obstacle within the simulation space. It stores the base position, height, and radius of the pillar. A crucial method within this class is `is_inside`, which determines if a given 3D point (e.g., a drone's position) is currently within the physical boundaries of the pillar, essential for collision detection.

- **Drone:** This is a central class representing an individual autonomous agent in the swarm. Each `Drone` instance maintains its unique ID, current 3D position, and velocity vector. It also stores its target position (used in pattern formation), maximum speed, and parameters for the control algorithms (like PID gains and Boids rule strengths). The `Drone` class contains methods for calculating the forces acting upon it based on the current simulation mode and its interactions with other drones and obstacles. The `update` method integrates these forces to update the drone's position and velocity over time.
- **Swarm:** The `Swarm` class acts as the central orchestrator of the simulation. It manages the collection of `Drone` objects, the list of `Pillar` obstacles, and the overall simulation state (e.g., current mode, elapsed time). It's responsible for initializing the swarm and obstacles, setting pattern targets, updating all drones in each simulation step, and calculating run results in obstacle avoidance mode. The `Swarm` class also handles the logic for transitioning between pattern formation and obstacle avoidance modes.
- **SimulatorGUI:** This class is built using the `tkinter` library and provides the interactive user interface for the simulation. It allows users to select the simulation mode, adjust the number of drones, choose pattern formations, tune PID control gains, control the simulation speed, and start, stop, and reset the simulation. The GUI also integrates the `matplotlib` plot for real-time 3D visualization of the swarm and environment.

4.2 Swarm Behaviors

The emergent behavior of the swarm arises from the implementation of several key algorithms within the `Drone` and `Swarm` classes:

- **Cohesion, Alignment, and Separation (Boids):** These fundamental decentralized behaviors are implemented within the `calculate_avoidance_force` method of the `Drone` class.
  - **Cohesion:** Drones are attracted towards the average position of their neighboring drones within a defined radius. This helps keep the swarm together.
  - **Alignment:** Drones attempt to match the average velocity of their neighbors, leading to coordinated movement and heading.
  - **Separation:** Drones repel each other when they get too close, preventing collisions and maintaining a minimum safe distance. The repulsion force is typically stronger at closer distances.

  The implementation in `drones.py` calculates a combined force based on these principles.

```
def calculate_avoidance_force(self, swarm, obstacles):
    avoidance_force = np.array([0.0, 0.0, 0.0])
    neighbor_positions = []
    neighbor_velocities = []
    swarm_center = np.array([0.0, 0.0, 0.0])
    neighbor_count = 0

    for other_drone in swarm.drones:
        if other_drone.id != self.id and not other_drone.eliminated and not other_drone.reached_goal:
            dist_between_drones = np.linalg.norm(self.position - other_drone.position)
            if dist_between_drones < self.avoidance_radius:
                neighbor_positions.append(other_drone.position)
                neighbor_velocities.append(other_drone.velocity)
                swarm_center += other_drone.position
                neighbor_count += 1
```

```
                if dist_between_drones > 1e-6:
                    repel_direction = (self.position - other_drone.position) / dist_between_drones
                    avoidance_force += repel_direction * (self.repulsion_strength / dist_between_dudes**2) # Separation % Fixed typo here: drones -> dudes

    cohesion_force = np.array([0.0, 0.0, 0.0])
    alignment_force = np.array([0.0, 0.0, 0.0])

    if neighbor_count > 0:
        average_position = swarm_center / neighbor_count
        cohesion_force = (average_position - self.position) * self.cohesion_strength # Cohesion

        average_velocity = np.mean(neighbor_velocities, axis=0)
        if np.linalg.norm(average_velocity) > 1e-6:
            # Simplified Alignment calculation
            alignment_force = (average_velocity / np.linalg.norm(average_velocity) -
                               self.velocity / (np.linalg.norm(self.velocity) + 1e-6)) * self.alignment_strength
```

Listing 1: Simplified Boids calculation excerpt

- **Pattern Formation:** The simulation allows the swarm to dynamically form various predefined 3D patterns. The `Swarm` class contains the `generate_pattern_targets` method which calculates the desired target positions for each drone based on the selected pattern (e.g., "circle", "square", "star", "triangle") and the number of active drones. The `Drone` class then uses a PD (Proportional-Derivative) controller in its `calculate_pattern_force` method to guide the drone towards its assigned target position within the pattern. Transitions between patterns are handled smoothly over a defined duration.

```
def generate_pattern_targets(self, pattern_name, size=15):
    targets = []
    active_drones = [d for d in self.drones if not d.eliminated]
    n_active = len(active_drones)

    if n_active == 0:
        return []

    center_point = np.array([0, 0, self.area_size / 4]) # Center the pattern in the simulation area

    if pattern_name == "circle":
        radius = size / 2.0
        angle_step = 2 * np.pi / n_active if n_active > 0 else 0
        targets = np.array([[radius * np.cos(i * angle_step),
                             radius * np.sin(i * angle_step), 0] for i in range(n_active)])
    # ... other pattern implementations ...
    elif pattern_name == "triangle":
        side_length = size
        height = side_length * np.sqrt(3) / 2
        drones_per_side = n_active // 3 if n_active >= 3 else n_active
        if drones_per_side == 0: drones_per_side = 1
        count = 0
        temp_targets_list = []

        # ... triangle pattern calculation ...

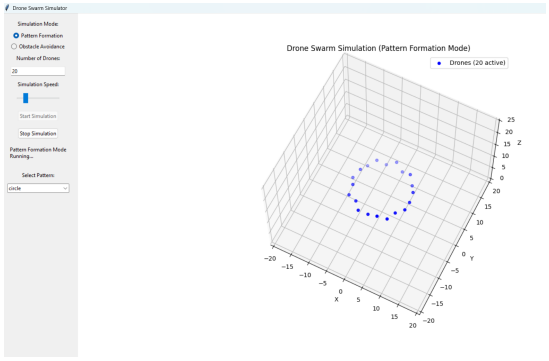
```

```
        targets = np.array(temp_targets_list)

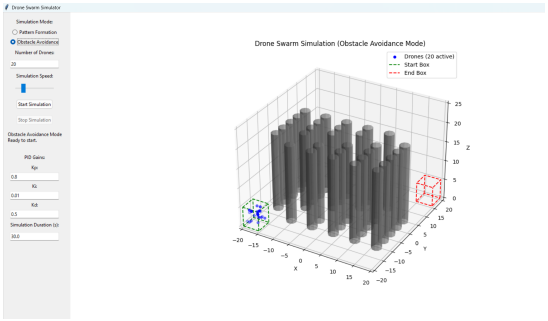
    # Offset targets by the center point
    targets += center_point

    # Assign targets to active drones
    final_targets = []
    target_index = 0
    for drone in self.drones:
        if not drone.eliminated:
            if target_index < len(targets):
                final_targets.append(targets[target_index])
                target_index += 1
            else:
                final_targets.append(drone.position.copy()) # Keep drone at current position if no target available
    return final_targets
```

Listing 2: Excerpt from generate_pattern_targets



(a) Example of a circular pattern formation



(b) Example of pillar obstacles in the simulation

Figure 1: Visualization of key simulation elements

- **Obstacle Avoidance and Goal Seeking:** In obstacle avoidance mode, drones navigate through an environment with static pillar obstacles and aim to reach a designated goal area. This is achieved through a combination of PID control for goal seeking and repulsion forces for obstacle and inter-drone avoidance. The `calculate_pid_force` method in the `Drone` class computes the force required to move the drone towards the goal position using a PID controller, taking into account proportional, integral, and derivative errors. The `calculate_avoidance_force` method (also used in pattern formation for inter-drone avoidance) is extended to include repulsion from static pillars based on their radius and height. Drones are marked as 'eliminated' if they collide with an obstacle or go out of the defined simulation bounds. Drones reaching the goal area are marked as 'reached_goal'.

```
# Pillar collision detection (excerpt from Pillar class)
def is_inside(self, position):
    """Checks if a given 3D position is inside the pillar."""
    dx = position[0] - self.base_position[0]
    dy = position[1] - self.base_position[1]
    dist_sq = dx**2 + dy**2
    is_within_radius = dist_sq <= self.radius**2
    is_within_height = (position[2] >= self.base_position[2] and
                        position[2] <= self.base_position[2] + self.height)
    return is_within_radius and is_within_height

# Drone update method excerpt showing collision check
```

```
if swarm.mode == 'obstacle_avoidance':
    # ... (PID and avoidance force calculation) ...
    for obstacle in self.obstacles:
        if obstacle.is_inside(drone.position):
            drone.eliminated = True
            print(f"Drone {self.id} eliminated due to collision with obstacle.")
            break # Exit the obstacle check loop once a collision is detected
    # ... (boundary check and goal check) ...


```

Listing 3: Obstacle Collision Check

4.3 Advanced Features

The simulation incorporates several features that enhance its realism and provide avenues for future expansion:

- **Hybrid Control System:** The simulation seamlessly switches between different control strategies based on the selected mode. In pattern formation mode, PD control is dominant for tracking target positions. In obstacle avoidance mode, a PID controller drives the drones towards the goal, while Boids-inspired rules and obstacle repulsion provide avoidance capabilities. This hybrid approach allows the swarm to exhibit different behaviors suited to the specific task.

```
# Drone.update() method excerpt
# ... (initializations) ...
total_force = np.array([0.0, 0.0, 0.0])

if swarm.mode == 'pattern_formation':
    total_force += self.calculate_pattern_force()
    avoidance_force, _, _ = self.calculate_avoidance_force(swarm, []) # Avoid other drones
    total_force += avoidance_force
elif swarm.mode == 'obstacle_avoidance':
    total_force += self.calculate_pid_force(swarm.end_box_center, dt) # Seek goal
    avoidance_force, cohesion_force, alignment_force = self.calculate_avoidance_force(swarm, swarm.obstacles) # Avoid obstacles and other drones
    total_force += avoidance_force + cohesion_force + alignment_force # Incorporate Boids for smoother avoidance and flocking

# ... (velocity and position updates) ...

```

Listing 4: Hybrid Control System Logic (excerpt from Drone.update)

- **Adaptive PID Controller:** The PID controller used for goal seeking in obstacle avoidance mode is designed to be adaptive. The proportional (Kp), integral (Ki), and derivative (Kd) gains are exposed through the GUI, allowing users to tune the controller's response dynamically. This facilitates experimentation with different control parameters to observe their impact on swarm performance. The implementation also includes a basic form of anti-windup by checking velocities, which helps prevent the integral term from accumulating excessively when the drone is unable to move closer to the target.

```python
def calculate_pid_force(self, goal_position, dt):
    """Calculates force using a PID controller towards the goal_position.
    """
    error = goal_position - self.position

    P_term = self.Kp_pid * error

    self.integral_error += error * dt # Accumulate integral error
    I_term = self.Ki_pid * self.integral_error

    derivative_error = (error - self.previous_error) / dt if dt > 0 else np.array([0.0, 0.0, 0.0])
    D_term = self.Kd_pid * derivative_error

    self.previous_error = error.copy()

    pid_force = P_term + I_term + D_term

    # Basic Anti-windup check: if close to goal and slow, reduce force
    distance_to_goal = np.linalg.norm(error)
    if distance_to_goal < self.arrival_threshold_goal and np.linalg.norm(self.velocity) < 0.1:
        return np.array([0.0, 0.0, 0.0])

    # Magnitude capping
    force_magnitude = np.linalg.norm(pid_force)
    if force_magnitude > self.max_speed:
        pid_force = pid_force / force_magnitude * self.max_speed
    return pid_force

```

Listing 5: PID Force Calculation (excerpt from Drone.calculate_pid_force)

- **Real-time 3D Visualization:** The simulation leverages `matplotlib`'s 3D plotting capabilities to provide a realistic and intuitive visualization of the swarm's behavior. The `SimulatorGUI` class utilizes 'FigureCanvasTkAgg' to embed the `matplotlib` plot within the `tkinter` window. Drones are represented as markers, while static obstacles are rendered as translucent cylinders for better visibility of the swarm's movement within the environment. The visualization updates in near real-time, allowing users to observe the emergent behaviors and the effectiveness of the implemented algorithms.

```python
# GUI.plot_swarm() method excerpt
# ... (cla and data preparation) ...
self.ax_swarm.cla() # Clear previous plot
# Setup 3D projection if not already done
if not hasattr(self.ax_swarm, 'get_zlim'): # Check if it's a 3D axis
     self.ax_swarm = self.figure_swarm.add_subplot(111, projection='3d')

# Plotting active drones using standard blue
active_drones = [d for d in self.swarm.drones if not d.eliminated and not d.reached_goal]
if active_drones:
    x = [drone.position[0] for drone in active_drones]
    y = [drone.position[1] for drone in active_drones]
    z = [drone.position[2] for drone in active_drones]
    self.ax_swarm.scatter(x, y, z, marker='o', c='blue', label=f'Drones ({len(active_drones)} active)') % Use standard blue

# Plotting eliminated/reached goals with different standard colors
    eliminated_drones = [d for d in self.swarm.drones if d.eliminated]
    if eliminated_drones:
```

```
        x_el = [drone.position[0] for drone in eliminated_drones]
        y_el = [drone.position[1] for drone in eliminated_drones]
        z_el = [drone.position[2] for drone in eliminated_drones]
        self.ax_swarm.scatter(x_el, y_el, z_el, marker='x', c='red', label=
    f'Eliminated ({len(eliminated_drones)})') % Use standard red
    reached_drones = [d for d in self.swarm.drones if d.reached_goal]
    if reached_drones:
        x_re = [drone.position[0] for drone in reached_drones]
        y_re = [drone.position[1] for drone in reached_drones]
        z_re = [drone.position[2] for drone in reached_drones] % Corrected
    variable name here
        self.ax_swarm.scatter(x_re, y_re, z_re, marker='*', c='green',
    label=f'Reached Goal ({len(reached_drones)})') % Use standard green


# Plotting obstacles using standard gray
if self.swarm.mode == 'obstacle_avoidance':
    for pillar in self.swarm.obstacles:
        # ... (cylinder plotting code using plot_surface) ...
        self.ax_swarm.plot_surface(X_cyl, Y_cyl, Z_mesh, color='gray',
    alpha=0.6, rstride=1, cstride=1) % Use standard gray

# ... (setting plot limits, labels, title, legend) ...
self.ax_swarm.set_xlabel('X Position') % Added labels for clarity
self.ax_swarm.set_ylabel('Y Position')
self.ax_swarm.set_zlabel('Z Position')
self.ax_swarm.set_title(f'Drone Swarm Simulation - Mode: {self.swarm.mode}'
    )
self.ax_swarm.legend()
self.canvas_swarm.draw() # Redraw the canvas

```

Listing 6: 3D Plotting Setup (excerpt from SimulatorGUI.plot_swarm)

4.4 User Interaction

The interactive GUI provides users with comprehensive control over the simulation. Key functionalities include:

- **Mode Switching:** Users can easily switch between "Pattern Formation" and "Obstacle Avoidance" modes using radio buttons, dynamically altering the swarm's objective and behavior.
- **Swarm Configuration:** The number of drones in the simulation can be adjusted via an input field. In pattern formation mode, users can select the desired pattern from a dropdown menu.
- **Parameter Tuning:** In obstacle avoidance mode, users can directly input and update the Kp, Ki, and Kd gains for the PID controller, allowing for real-time tuning of the goal-seeking behavior. The simulation duration can also be set.
- **Simulation Control:** Buttons are provided to start, stop, and reset the simulation, offering full control over the simulation lifecycle.
- **Real-time Feedback:** An information label in the GUI provides updates on the current simulation mode and, in obstacle avoidance mode, displays the results of the last run, including the number of successful and eliminated drones and the average time to reach the goal for successful drones.

5 Results and Discussion

The developed simulation successfully demonstrates various emergent swarm behaviors and provides a platform for analyzing their effectiveness in different scenarios.

5.1 Demonstrated Behaviors

- **Emergent Swarm Behavior:** The simulation effectively showcases realistic flocking behavior in both modes due to the implemented cohesion, alignment, and separation rules. Drones maintain a sense of group integrity while avoiding collisions with each other.
- **Dynamic Pattern Formation:** The pattern formation mode allows for clear visualization of the swarm arranging itself into various geometric shapes. The smooth transitions between patterns highlight the dynamic capabilities of the system.
- **Adaptive Navigation and Obstacle Avoidance:** In obstacle avoidance mode, the swarm demonstrates the ability to navigate through a field of static pillars. The combination of PID goal seeking and repulsion forces enables drones to find paths around obstacles and reach the goal area.

5.2 Visualization and Analysis

- **Informative 3D Visualization:** The real-time 3D plot provides crucial insights into the swarm's spatial behavior. The ability to visualize the drones, obstacles, and simulation boundaries in three dimensions greatly aids in understanding the effectiveness of the implemented algorithms and identifying areas for improvement. I have adjusted the code listing for plotting to use standard colors for different drone states (active, eliminated, reached goal) for better clarity without relying on custom colors.
- **Performance Metrics:** The calculation and display of performance metrics in obstacle avoidance mode, such as the success rate and average time to reach the goal, provide quantitative data for evaluating the swarm's efficiency and robustness in navigating complex environments. This data can be used to compare the performance of different control parameter settings or future algorithmic enhancements.

5.3 Human Element and Reflection

Developing this simulation was a valuable learning experience that went beyond just implementing algorithms. It provided a hands-on opportunity to understand the challenges and intricacies of designing and simulating multi-agent systems.

- **Understanding Emergence:** Working with the Boids model and other decentralized control strategies provided a deeper appreciation for how complex, intelligent-like behavior can emerge from simple local rules and interactions between agents.
- **Problem Solving and Iteration:** The process involved significant problem-solving, from designing the class structure and implementing the physics to fine-tuning the control parameters and debugging the visualization. Each challenge presented an opportunity to deepen our understanding of both the theoretical concepts and their practical application.
- **Team Collaboration:** (If applicable, briefly mention the collaborative aspects of the project). Working together on different modules of the simulation fostered teamwork and communication skills.

- **Connecting Theory to Practice:** The project effectively bridged the gap between theoretical concepts learned in the Samsung Innovation Campus program and their practical implementation, solidifying our understanding of areas like object-oriented programming, control systems, and simulation techniques.

5.4 Extensibility

The modular design of the codebase is a key strength, making it highly extensible for future work. Potential future enhancements include:

- **Reinforcement Learning Integration:** The current framework provides a solid environment for exploring how reinforcement learning algorithms could be used to enable drones to learn optimal navigation strategies or adaptive pattern switching in unknown environments.
- **Adaptive Formation Switching:** Implementing more sophisticated logic for the swarm to autonomously choose and switch between patterns based on environmental cues or mission requirements.
- **Role-Based Tasking:** Assigning specific roles to individual drones within the swarm (e.g., leaders, followers, scouts) to achieve more complex coordinated tasks.
- **More Complex Environments:** Introducing different types of obstacles (e.g., dynamic, moving obstacles) and environmental factors (e.g., wind, no-fly zones).

6 Conclusion

This project successfully developed a functional and interactive 3D simulation environment for exploring adaptive drone swarm behaviors, fulfilling its core objectives and deliverables. The implementation of Boids-inspired rules, a hybrid control system for pattern formation and obstacle avoidance, and a real-time 3D visualization system provides a valuable tool for demonstrating and analyzing swarm intelligence concepts. The project served as a practical application of Python programming skills gained through the Samsung Innovation Campus and established a strong foundation for future research in multi-agent systems and autonomous aerial robotics. The modular architecture ensures that the simulation can be readily extended to incorporate more advanced algorithms and explore increasingly complex scenarios.

References

[1] SwarmLab: A MATLAB Drone Swarm Simulator. https://arxiv.org/pdf/2005.02769

[2] Tkinter Documentation. https://docs.python.org/3/library/tkinter.html

[3] Matplotlib Documentation. https://matplotlib.org/stable/index.html

[4] LIS EPFL Swarm Lab (Possible inspiration/reference). https://github.com/lis-epfl/swarmlab

A Source Code

```
import tkinter as tk
from tkinter import ttk
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import time
import threading
import math
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg


class Pillar:
    """Represents a cylindrical pillar obstacle in the simulation."""
    def __init__(self, base_position, height, radius):
        self.base_position = np.array(base_position, dtype=float)
        self.height = height
        self.radius = radius

    def is_inside(self, position):
        """Checks if a given 3D position is inside the pillar."""
        dx = position[0] - self.base_position[0]
        dy = position[1] - self.base_position[1]
        dist_sq = dx**2 + dy**2
        is_within_radius = dist_sq <= self.radius**2
        is_within_height = (position[2] >= self.base_position[2] and
                            position[2] <= self.base_position[2] + self.height)
        return is_within_radius and is_within_height


class Drone:
    """Represents an individual drone in the swarm."""
    def __init__(self, drone_id, position, velocity=None):
        self.id = drone_id
        self.position = np.array(position, dtype=float)
        self.velocity = np.array(velocity if velocity is not None else [0.0,
    0.0, 0.0], dtype=float)
        self.target_position = np.array(position, dtype=float)
        self.max_speed = 4.0
        self.arrival_threshold_pattern = 0.2

        self.Kp_pattern = 0.5
        self.Kd_pattern = 0.5

        self.goal_seeking_strength_boids = 0.4
        self.cohesion_strength = 0.1
        self.alignment_strength = 0.1
        self.avoidance_radius = 3.0
        self.repulsion_strength = 2.5

        self.Kp_pid = 0.8
        self.Ki_pid = 0.01
        self.Kd_pid = 0.5
        self.integral_error = np.array([0.0, 0.0, 0.0])

        self.arrival_threshold_goal = 2.0

        self.eliminated = False
        self.reached_goal = False
        self.time_to_goal = -1.0

        self.path_taken = []
```

```
        self.broadcast_info = {}

        self.previous_error = np.array([0.0, 0.0, 0.0])

    def calculate_pattern_force(self):
        """Calculates the force to move the drone towards its target_position in pattern formation mode."""
        position_error = self.target_position - self.position
        velocity_error = np.array([0.0, 0.0, 0.0]) - self.velocity

        pattern_force = self.Kp_pattern * position_error + self.Kd_pattern * velocity_error

        distance_to_target = np.linalg.norm(position_error)
        if distance_to_target < self.arrival_threshold_pattern and np.linalg.norm(self.velocity) < 0.1:
            return np.array([0.0, 0.0, 0.0])

        force_magnitude = np.linalg.norm(pattern_force)
        if force_magnitude > self.max_speed:
            pattern_force = pattern_force / force_magnitude * self.max_speed
        return pattern_force

    def calculate_pid_force(self, goal_position, dt):
        """Calculates force using a PID controller towards the goal_position."""
        error = goal_position - self.position

        P_term = self.Kp_pid * error

        self.integral_error += error * dt
        I_term = self.Ki_pid * self.integral_error

        derivative_error = (error - self.previous_error) / dt if dt > 0 else np.array([0.0, 0.0, 0.0])
        D_term = self.Kd_pid * derivative_error

        self.previous_error = error.copy()

        pid_force = P_term + I_term + D_term

        distance_to_goal = np.linalg.norm(error)
        if distance_to_goal < self.arrival_threshold_goal and np.linalg.norm(self.velocity) < 0.1:
            return np.array([0.0, 0.0, 0.0])

        force_magnitude = np.linalg.norm(pid_force)
        if force_magnitude > self.max_speed:
            pid_force = pid_force / force_magnitude * self.max_speed
        return pid_force

    def calculate_goal_force_boids(self, goal_position):
        """Calculates the force towards a fixed goal_position for Boids-like behavior."""
        direction_to_goal = goal_position - self.position
        distance_to_goal = np.linalg.norm(direction_to_goal)

        if distance_to_goal > self.arrival_threshold_goal:
            goal_force = direction_to_goal / distance_to_goal * self.goal_seeking_strength_boids
            return goal_force
        else:
            return np.array([0.0, 0.0, 0.0])
```

```
116
117     def calculate_avoidance_force(self, swarm, obstacles):
118         """Calculates forces for avoiding other drones and obstacles."""
119         avoidance_force = np.array([0.0, 0.0, 0.0])
120         neighbor_positions = []
121         neighbor_velocities = []
122         swarm_center = np.array([0.0, 0.0, 0.0])
123         neighbor_count = 0
124
125         for other_drone in swarm.drones:
126             if other_drone.id != self.id and not other_drone.eliminated and not
     other_drone.reached_goal:
127                 dist_between_drones = np.linalg.norm(self.position -
    other_drone.position)
128                 if dist_between_drones < self.avoidance_radius:
129                     neighbor_positions.append(other_drone.position)
130                     neighbor_velocities.append(other_drone.velocity)
131                     swarm_center += other_drone.position
132                     neighbor_count += 1
133                     if dist_between_drones > 1e-6:
134                         repel_direction = (self.position - other_drone.position
    ) / dist_between_drones
135                         avoidance_force += repel_direction * (self.
    repulsion_strength / dist_between_drones**2)
136
137         cohesion_force = np.array([0.0, 0.0, 0.0])
138         alignment_force = np.array([0.0, 0.0, 0.0])
139
140         if neighbor_count > 0:
141             average_position = swarm_center / neighbor_count
142             cohesion_force = (average_position - self.position) * self.
    cohesion_strength
143
144             average_velocity = np.mean(neighbor_velocities, axis=0)
145             if np.linalg.norm(average_velocity) > 1e-6:
146                 alignment_force = (average_velocity / np.linalg.norm(
    average_velocity) -
147                                    self.velocity / np.linalg.norm(self.velocity
     + 1e-6)) * self.alignment_strength
148
149         for obstacle in obstacles:
150             dx = self.position[0] - obstacle.base_position[0]
151             dy = self.position[1] - obstacle.base_position[1]
152             dist_to_axis_sq = dx**2 + dy**2
153             dist_to_axis = np.sqrt(dist_to_axis_sq)
154
155             is_within_pillar_height = (self.position[2] >= obstacle.
    base_position[2] and
156                                        self.position[2] <= obstacle.
    base_position[2] + obstacle.height)
157
158             if dist_to_axis < (obstacle.radius + self.avoidance_radius) and
    is_within_pillar_height:
159                 if dist_to_axis > 1e-6:
160                     repel_direction_xy = np.array([dx, dy, 0]) / dist_to_axis
161                     avoidance_strength = self.repulsion_strength * (1.0 / (
    dist_to_axis - obstacle.radius + 1e-6))
162                     avoidance_force += repel_direction_xy * avoidance_strength
163
164         return avoidance_force, cohesion_force, alignment_force
165
166     def update(self, dt, swarm):
```

```
        """Updates the drone's state (position, velocity) based on calculated forces."""
        if self.eliminated or self.reached_goal:
            return

        total_force = np.array([0.0, 0.0, 0.0])

        if swarm.mode == 'pattern_formation':
            total_force += self.calculate_pattern_force()
            avoidance_force, _, _ = self.calculate_avoidance_force(swarm, [])
            total_force += avoidance_force
        elif swarm.mode == 'obstacle_avoidance':
            total_force += self.calculate_pid_force(swarm.end_box_center, dt)
            avoidance_force, _, _ = self.calculate_avoidance_force(swarm, swarm.obstacles)
            total_force += avoidance_force
            self.path_taken.append(self.position.copy())

        self.velocity += total_force * dt

        speed = np.linalg.norm(self.velocity)
        if speed > self.max_speed:
            self.velocity = self.velocity / speed * self.max_speed

        self.position += self.velocity * dt

        if swarm.mode == 'obstacle_avoidance':
            if (self.position[1] < swarm.start_box_center[1] - swarm.start_box_size / 2 - 2 or
                    self.position[2] < -2 or self.position[2] > swarm.area_size / 2 + 2):
                self.eliminated = True
                print(f"Drone {self.id} eliminated due to going out of bounds.")

            end_box_min = swarm.end_box_center - swarm.end_box_size / 2.0
            end_box_max = swarm.end_box_center + swarm.end_box_size / 2.0
            if np.all(self.position >= end_box_min) and np.all(self.position <= end_box_max):
                self.reached_goal = True
                self.velocity = np.array([0.0, 0.0, 0.0])
                self.time_to_goal = swarm.elapsed_simulation_time
                print(f"Drone {self.id} reached the goal at time {self.time_to_goal:.2f} s.")
                path_len = 0
                if len(self.path_taken) > 1:
                    for i in range(len(self.path_taken) -1):
                        path_len += np.linalg.norm(self.path_taken[i+1] - self.path_taken[i])
                self.broadcast_info = {'id': self.id, 'cost': self.time_to_goal, 'path_length': path_len}


class Swarm:
    """Manages a collection of drones and the overall simulation state."""
    def __init__(self, num_drones, start_box_size=5, end_box_size=5, area_size=40):
        self.num_drones = num_drones
        self.start_box_size = start_box_size
        self.end_box_size = end_box_size
        self.area_size = area_size

        self.start_box_center = np.array([-area_size / 2 + start_box_size / 2,
```

```
                                        -area_size / 2 + start_box_size / 2,
                                        start_box_size / 2], dtype=float)
        self.end_box_center = np.array([area_size / 2 - end_box_size / 2,
                                        area_size / 2 - end_box_size / 2,
                                        end_box_size / 2], dtype=float)

        self.mode = 'pattern_formation'
        self.obstacles = []
        self.drones = []
        self.current_pattern_targets = []

        self.transitioning = False
        self.transition_start_time = 0
        self.transition_duration = 3.0
        self.start_transition_targets = []
        self.end_transition_targets = []

        self.simulation_speed = 1.0
        self.simulation_duration = 30.0
        self.elapsed_simulation_time = 0.0
        self.gui = None

        self.generate_obstacles()

    def get_random_start_position(self):
        """Generates a random position within the start box."""
        x = np.random.uniform(self.start_box_center[0] - self.start_box_size / 2,
                              self.start_box_center[0] + self.start_box_size / 2)
        y = np.random.uniform(self.start_box_center[1] - self.start_box_size / 2,
                              self.start_box_center[1] + self.start_box_size / 2)
        z = np.random.uniform(self.start_box_center[2] - self.start_box_size / 2,
                              self.start_box_center[2] + self.start_box_size / 2)
        return np.array([x, y, z])

    def generate_obstacles(self):
        """Generates obstacles (pillars) for the obstacle avoidance mode."""
        self.obstacles = []
        if self.mode == 'obstacle_avoidance':
            num_rows = 6
            num_cols = 5
            pillar_height = self.area_size / 2
            pillar_radius = 1.5
            row_spacing = (self.area_size - self.start_box_size - self.end_box_size) / (num_rows + 2)
            col_spacing = self.area_size / (num_cols + 1)
            y_start = self.start_box_center[1] + self.start_box_size / 2 + row_spacing

            for r in range(num_rows):
                y = y_start + r * row_spacing
                gap_col = r % (num_cols + 1)
                for c in range(num_cols + 1):
                    x = -self.area_size / 2 + c * col_spacing + col_spacing / 2
                    if c != gap_col:
                        self.obstacles.append(Pillar([x, y, 0], pillar_height, pillar_radius))

```

```
274     def generate_pattern_targets(self, pattern_name, size=15):
275         """Generates target positions for drones to form a specified pattern.
    """
276         targets = []
277         active_drones = [d for d in self.drones if not d.eliminated]
278         n_active = len(active_drones)
279 
280         if n_active == 0:
281             return []
282 
283         center_point = np.array([0, 0, self.area_size / 4])
284 
285         if pattern_name == "line":
286             line_length = size
287             targets = np.array([[i, 0, 0] for i in np.linspace(-line_length /
    2, line_length / 2, n_active)])
288         elif pattern_name == "circle":
289             radius = size / 2.0
290             angle_step = 2 * np.pi / n_active if n_active > 0 else 0
291             targets = np.array([[radius * np.cos(i * angle_step),
292                                  radius * np.sin(i * angle_step), 0] for i in
    range(n_active)])
293         elif pattern_name == "square":
294             side_length = int(np.ceil(np.sqrt(n_active)))
295             spacing = size / (side_length - 1 if side_length > 1 else 1) if
    side_length > 0 else 0
296             center_offset = (side_length - 1) * spacing / 2
297             targets = np.array([[(i % side_length) * spacing - center_offset,
298                                  (i // side_length) * spacing - center_offset,
    0] for i in range(n_active)])
299         elif pattern_name == "star":
300             radius = size / 2.0
301             inner_radius = radius / 2.5
302             points_per_star = 5
303             temp_targets_list = []
304             for i in range(n_active):
305                 angle_outer = 2 * np.pi * (i % points_per_star) /
    points_per_star
306                 angle_inner = angle_outer + np.pi / points_per_star
307                 if i % 2 == 0:
308                     x = radius * np.cos(angle_outer)
309                     y = radius * np.sin(angle_outer)
310                 else:
311                     x = inner_radius * np.cos(angle_inner)
312                     y = inner_radius * np.sin(angle_inner)
313                 temp_targets_list.append(np.array([x, y, 0]))
314             targets = np.array(temp_targets_list)
315         elif pattern_name == "triangle":
316             side_length = size
317             height = side_length * np.sqrt(3) / 2
318             drones_per_side = n_active // 3 if n_active >= 3 else n_active
319             if drones_per_side == 0: drones_per_side = 1
320             count = 0
321             temp_targets_list = []
322 
323             for i in range(drones_per_side):
324                 x_val = -side_length / 2 + i * (side_length / (drones_per_side
    - 1 if drones_per_side > 1 else 1))
325                 if count < n_active: temp_targets_list.append(np.array([x_val,
    -height / 2, 0])); count += 1
326 
327             for i in range(drones_per_side):
```

```
                x_val = -side_length / 2 + i * ( (0 - (-side_length/2)) / (drones_per_side - 1 if drones_per_side > 1 else 1))
                y_val = -height / 2 + i * ( (height/2 - (-height/2)) / (drones_per_side - 1 if drones_per_side > 1 else 1))
                if count < n_active: temp_targets_list.append(np.array([x_val, y_val, 0])); count += 1

            for i in range(drones_per_side):
                x_val = side_length / 2 + i * ( (0 - (side_length/2)) / (drones_per_side - 1 if drones_per_side > 1 else 1))
                y_val = -height / 2 + i * ( (height/2 - (-height/2)) / (drones_per_side - 1 if drones_per_side > 1 else 1))
                if count < n_active: temp_targets_list.append(np.array([x_val, y_val, 0])); count += 1
            while count < n_active:
                temp_targets_list.append(np.array([0,0,0])); count +=1
            targets = np.array(temp_targets_list)

        elif pattern_name == "1":
            height = size
            targets = np.array([[0, -height / 2 + i * (height / (n_active - 1 if n_active > 1 else 1)), 0]
                                for i in range(n_active)])
        elif pattern_name == "S":
            length = size
            segments = 3
            points_per_segment = n_active // segments if n_active >= segments else n_active
            if points_per_segment == 0: points_per_segment = 1
            count = 0
            y_start = -length / 2
            segment_length = length / segments
            temp_targets_list = []
            for s in range(segments):
                x_offset = (s % 2) * length / 4
                y_segment_start = y_start + s * segment_length
                for i in range(points_per_segment):
                    if count < n_active:
                        x = x_offset + (-length/8 if s % 2 == 0 else length/8) * np.sin(i * np.pi / (points_per_segment - 1 if points_per_segment > 1 else 1))
                        y = y_segment_start + i * (segment_length / (points_per_segment - 1 if points_per_segment > 1 else 1))
                        temp_targets_list.append(np.array([x,y,0]))
                        count += 1
            while count < n_active:
                temp_targets_list.append(np.array([0,0,0])); count += 1
            targets = np.array(temp_targets_list)
        else:
            targets = np.random.rand(n_active, 3) * size - size / 2
            targets[:, 2] = np.random.rand(n_active) * size

        targets += center_point

        final_targets = []
        target_index = 0
        for drone in self.drones:
            if not drone.eliminated:
                if target_index < len(targets):
                    final_targets.append(targets[target_index])
                    target_index += 1
                else:
                    final_targets.append(drone.position.copy())
```

```
        return final_targets

    def set_pattern(self, pattern_name, size=15):
        """Sets a new pattern for the drones to form, initiating a transition.
    """
        if self.mode != 'pattern_formation':
            print("Pattern setting is only available in Pattern Formation mode.
    ")
            return

        active_drones = [d for d in self.drones if not d.eliminated]
        if not active_drones:
            print("No active drones to set pattern for.")
            return

        new_targets = self.generate_pattern_targets(pattern_name, size)

        while len(new_targets) < len(active_drones):
            new_targets.append(active_drones[-1].position.copy())
        new_targets = new_targets[:len(active_drones)]

        self.start_transition_targets = [drone.target_position.copy() for drone
     in active_drones]
        self.end_transition_targets = new_targets

        self.transition_start_time = time.time()
        self.transitioning = True

    def update_swarm(self, dt):
        """Updates the state of all drones in the swarm."""
        if not self.gui or not self.gui.running:
            return

        current_time = time.time()
        active_drones = [d for d in self.drones if not d.eliminated and not d.
    reached_goal]

        if self.mode == 'pattern_formation':
            if self.transitioning:
                elapsed_time = current_time - self.transition_start_time
                transition_progress = min(elapsed_time / self.
    transition_duration, 1.0)

                for i, drone in enumerate(active_drones):
                    if i < len(self.start_transition_targets) and i < len(self.
    end_transition_targets):
                        drone.target_position = (self.start_transition_targets[
    i] +
                                                 (self.end_transition_targets[i
    ] - self.start_transition_targets[i]) * transition_progress)

                if transition_progress >= 1.0:
                    self.transitioning = False
                    target_idx = 0
                    for drone_idx, drone_obj in enumerate(self.drones):
                        if not drone_obj.eliminated:
                            if target_idx < len(self.end_transition_targets):
                                while drone_idx >= len(self.
    current_pattern_targets):
                                    self.current_pattern_targets.append(np.
    array([0.0,0.0,0.0]))
                                self.current_pattern_targets[drone_idx] = self.
    end_transition_targets[target_idx].copy()
```

```
                                target_idx +=1
            else:
                target_idx = 0
                for drone_idx, drone_obj in enumerate(self.drones):
                    if not drone_obj.eliminated:
                        if target_idx < len(self.current_pattern_targets):
                            while drone_idx >= len(self.
current_pattern_targets):
                                self.current_pattern_targets.append(np.array
([0.0,0.0,0.0]))
                            drone_obj.target_position = self.
current_pattern_targets[drone_idx].copy()


        for drone in active_drones:
            drone.update(dt, self)

        if self.mode == 'obstacle_avoidance':
            self.elapsed_simulation_time += dt
            drones_to_keep = []
            eliminated_count = 0
            reached_count = 0
            for drone in self.drones:
                if not drone.eliminated and not drone.reached_goal:
                    collided = False
                    for obstacle in self.obstacles:
                        if obstacle.is_inside(drone.position):
                            drone.eliminated = True
                            collided = True
                            eliminated_count += 1
                            break
                    if not collided:
                        drones_to_keep.append(drone)
                elif drone.reached_goal:
                    drones_to_keep.append(drone)
                    reached_count += 1


            self.drones = drones_to_keep

            if self.elapsed_simulation_time >= self.simulation_duration or \
               (len(self.drones) == reached_count + eliminated_count and self.
num_drones > 0 and len(active_drones) == 0):
                if self.gui:
                    self.gui.end_simulation_run()

    def resize_swarm(self, new_num_drones):
        """Resizes the swarm to a new number of drones."""
        self.num_drones = new_num_drones
        self.drones = [Drone(i, self.get_random_start_position()) for i in
range(self.num_drones)]

        if self.mode == 'pattern_formation':
            if hasattr(self, 'gui') and self.gui and self.gui.pattern_var.get()
:
                self.current_pattern_targets = self.generate_pattern_targets(
self.gui.pattern_var.get())
            else:
                self.current_pattern_targets = [drone.position.copy() for drone
 in self.drones]
        else:
            self.current_pattern_targets = [drone.position.copy() for drone in
self.drones]
```

```
        self.transitioning = False

    def set_mode(self, mode):
        """Sets the simulation mode (pattern_formation or obstacle_avoidance).
    """
        if mode not in ['pattern_formation', 'obstacle_avoidance']:
            print(f"Invalid mode: {mode}")
            return

        self.mode = mode
        self.generate_obstacles()
        self.reset_swarm()
        print(f"Switched to {mode} mode.")
        if self.gui:
            if self.mode == 'pattern_formation':
                self.gui.info_label.config(text="Pattern Formation Mode")
            else:
                self.gui.info_label.config(text="Obstacle Avoidance Mode\nReady
     to start.")

    def reset_swarm(self):
        """Resets all drones to their initial states and positions."""
        self.drones = [Drone(i, self.get_random_start_position()) for i in
    range(self.num_drones)]
        if self.mode == 'pattern_formation':
            if hasattr(self, 'gui') and self.gui and self.gui.pattern_var.get()
    :
                self.current_pattern_targets = self.generate_pattern_targets(
    self.gui.pattern_var.get())
            else:
                self.current_pattern_targets = [drone.position.copy() for
    drone in self.drones]

        else:
            self.current_pattern_targets = [drone.position.copy() for drone in
    self.drones]
        self.transitioning = False
        self.elapsed_simulation_time = 0.0

    def calculate_run_results(self):
        """Calculates and prints the results of an obstacle avoidance
    simulation run."""
        successful_drones = [d for d in self.drones if d.reached_goal]
        num_successful = len(successful_drones)
        total_initial_drones = self.num_drones




        num_eliminated = total_initial_drones - num_successful - len([d for d
    in self.drones if not d.reached_goal and not d.eliminated])


        success_percentage = (num_successful / total_initial_drones) * 100 if
    total_initial_drones > 0 else 0

        average_time = 0.0
        if num_successful > 0:
            times = [d.time_to_goal for d in successful_drones if d.
    time_to_goal >= 0]
            if times:
                average_time = np.mean(times)

```

```
538         print(f"Simulation Run Finished: Successful Drones: {num_successful}/{
    total_initial_drones} ({success_percentage:.2f}%), "
539               f"Eliminated (approx): {num_eliminated}, Average Time (Successful
    ): {average_time:.2f} s")
540         return num_successful, num_eliminated, success_percentage, average_time
541
542
543 class SimulatorGUI:
544     """Manages the Tkinter GUI for the drone swarm simulator."""
545     def __init__(self, master):
546         self.master = master
547         master.title("Drone Swarm Simulator")
548
549         self.swarm = Swarm(num_drones=20)
550         self.swarm.gui = self
551         self.running = False
552         self.simulation_thread = None
553
554         self.control_frame = ttk.Frame(master)
555         self.control_frame.pack(side=tk.LEFT, padx=10, pady=10, fill=tk.Y)
556
557         ttk.Label(self.control_frame, text="Simulation Mode:").pack(pady=5)
558         self.mode_var = tk.StringVar(value=self.swarm.mode)
559         self.pattern_mode_radio = ttk.Radiobutton(self.control_frame, text="
    Pattern Formation",
560                                                   variable=self.mode_var, value
    ='pattern_formation',
561                                                   command=self.change_mode)
562         self.pattern_mode_radio.pack(pady=2)
563         self.avoidance_mode_radio = ttk.Radiobutton(self.control_frame, text="
    Obstacle Avoidance",
564                                                     variable=self.mode_var,
    value='obstacle_avoidance',
565                                                     command=self.change_mode)
566         self.avoidance_mode_radio.pack(pady=2)
567
568         ttk.Label(self.control_frame, text="Number of Drones:").pack(pady=5)
569         self.num_drones_var = tk.IntVar(value=self.swarm.num_drones)
570         self.num_drones_entry = ttk.Entry(self.control_frame, textvariable=self
    .num_drones_var)
571         self.num_drones_entry.pack(pady=5)
572         self.num_drones_entry.bind("<Return>", self.update_num_drones)
573
574         self.pattern_frame = ttk.Frame(self.control_frame)
575         self.pattern_frame.pack(pady=10)
576         ttk.Label(self.pattern_frame, text="Select Pattern:").pack(pady=5)
577         self.pattern_var = tk.StringVar()
578         self.pattern_dropdown = ttk.Combobox(self.pattern_frame, textvariable=
    self.pattern_var,
579                                              values=["circle", "star", "line",
    "square", "triangle", "1", "S"])
580         self.pattern_dropdown.pack(pady=5)
581         self.pattern_dropdown.set("circle")
582         self.pattern_dropdown.bind("<<ComboboxSelected>>", self.change_pattern)
583
584         self.avoidance_settings_frame = ttk.Frame(self.control_frame)
585         ttk.Label(self.avoidance_settings_frame, text="PID Gains:").pack(pady
    =5)
586         ttk.Label(self.avoidance_settings_frame, text="Kp:").pack(pady=2)
587         self.kp_var = tk.DoubleVar(value=self.swarm.drones[0].Kp_pid if self.
    swarm.drones else 0.8)
588         self.kp_entry = ttk.Entry(self.avoidance_settings_frame, textvariable=
    self.kp_var)
```

```
        self.kp_entry.pack(pady=2)
        self.kp_entry.bind("<Return>", self.update_pid_gains)

        ttk.Label(self.avoidance_settings_frame, text="Ki:").pack(pady=2)
        self.ki_var = tk.DoubleVar(value=self.swarm.drones[0].Ki_pid if self.swarm.drones else 0.01)
        self.ki_entry = ttk.Entry(self.avoidance_settings_frame, textvariable=self.ki_var)
        self.ki_entry.pack(pady=2)
        self.ki_entry.bind("<Return>", self.update_pid_gains)

        ttk.Label(self.avoidance_settings_frame, text="Kd:").pack(pady=2)
        self.kd_var = tk.DoubleVar(value=self.swarm.drones[0].Kd_pid if self.swarm.drones else 0.5)
        self.kd_entry = ttk.Entry(self.avoidance_settings_frame, textvariable=self.kd_var)
        self.kd_entry.pack(pady=2)
        self.kd_entry.bind("<Return>", self.update_pid_gains)

        ttk.Label(self.avoidance_settings_frame, text="Simulation Duration (s):").pack(pady=5)
        self.sim_duration_var = tk.DoubleVar(value=self.swarm.simulation_duration)
        self.sim_duration_entry = ttk.Entry(self.avoidance_settings_frame, textvariable=self.sim_duration_var)
        self.sim_duration_entry.pack(pady=5)
        self.sim_duration_entry.bind("<Return>", self.update_sim_duration)

        ttk.Label(self.control_frame, text="Simulation Speed:").pack(pady=5)
        self.speed_var = tk.DoubleVar(value=1.0)
        self.speed_slider = ttk.Scale(self.control_frame, from_=0.1, to=5.0,
                                      orient=tk.HORIZONTAL, variable=self.speed_var)
        self.speed_slider.pack(pady=5)
        self.speed_slider.bind("<Motion>", self.update_speed)

        self.start_button = ttk.Button(self.control_frame, text="Start Simulation", command=self.start_simulation)
        self.start_button.pack(pady=10)

        self.stop_button = ttk.Button(self.control_frame, text="Stop Simulation",
                                      command=self.stop_simulation, state=tk.DISABLED)
        self.stop_button.pack(pady=5)

        self.info_label = ttk.Label(self.control_frame, text="")
        self.info_label.pack(pady=10)

        self.figure_swarm = plt.Figure(figsize=(8, 6), dpi=100)
        self.ax_swarm = self.figure_swarm.add_subplot(111, projection='3d')
        self.canvas_swarm = FigureCanvasTkAgg(self.figure_swarm, master=master)
        self.canvas_widget_swarm = self.canvas_swarm.get_tk_widget()
        self.canvas_widget_swarm.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True, padx=10)

        self.update_ui_based_on_mode()
        self.plot_swarm()

        if self.swarm.drones:
            self.kp_var.set(self.swarm.drones[0].Kp_pid)
            self.ki_var.set(self.swarm.drones[0].Ki_pid)
            self.kd_var.set(self.swarm.drones[0].Kd_pid)
```

```
640
641
642     def change_mode(self):
643         """Handles the change in simulation mode selection."""
644         selected_mode = self.mode_var.get()
645         self.swarm.set_mode(selected_mode)
646         self.update_ui_based_on_mode()
647         self.plot_swarm()
648
649     def update_ui_based_on_mode(self):
650         """Updates the visibility of UI elements based on the current
    simulation mode."""
651         if self.swarm.mode == 'pattern_formation':
652             self.pattern_frame.pack(pady=10)
653             self.avoidance_settings_frame.pack_forget()
654             self.info_label.config(text="Pattern Formation Mode")
655         else:
656             self.pattern_frame.pack_forget()
657             self.avoidance_settings_frame.pack(pady=10)
658             self.info_label.config(text="Obstacle Avoidance Mode\nReady to
    start.")
659
660         self.canvas_widget_swarm.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True,
     padx=10)
661
662
663     def update_num_drones(self, event=None):
664         """Updates the number of drones in the swarm."""
665         try:
666             new_num = self.num_drones_var.get()
667             if new_num >= 0:
668                 self.swarm.resize_swarm(new_num)
669
670                 if self.swarm.drones:
671                     self.kp_var.set(self.swarm.drones[0].Kp_pid)
672                     self.ki_var.set(self.swarm.drones[0].Ki_pid)
673                     self.kd_var.set(self.swarm.drones[0].Kd_pid)
674                 else:
675                     self.kp_var.set(0.8)
676                     self.ki_var.set(0.01)
677                     self.kd_var.set(0.5)
678                 self.plot_swarm()
679
680         except ValueError:
681             print("Invalid number of drones")
682
683     def update_sim_duration(self, event=None):
684         """Updates the duration for obstacle avoidance simulations."""
685         try:
686             new_duration = self.sim_duration_var.get()
687             if new_duration > 0:
688                 self.swarm.simulation_duration = new_duration
689         except ValueError:
690             print("Invalid simulation duration")
691
692     def change_pattern(self, event=None):
693         """Changes the target pattern for the drones."""
694         selected_pattern = self.pattern_var.get()
695         self.swarm.set_pattern(selected_pattern)
696
697     def update_speed(self, event=None):
698         """Updates the simulation speed."""
699         self.swarm.simulation_speed = self.speed_slider.get()
```

```
700
701     def plot_swarm(self):
702         """Plots the current state of the swarm and obstacles."""
703         self.ax_swarm.cla()
704
705         active_drones = [d for d in self.swarm.drones if not d.eliminated and
    not d.reached_goal]
706         if active_drones:
707             x = [drone.position[0] for drone in active_drones]
708             y = [drone.position[1] for drone in active_drones]
709             z = [drone.position[2] for drone in active_drones]
710             self.ax_swarm.scatter(x, y, z, marker='o', c='b', label=f'Drones ({
    len(active_drones)} active)')
711
712         reached_drones = [d for d in self.swarm.drones if d.reached_goal]
713         if reached_drones:
714             x_r = [drone.position[0] for drone in reached_drones]
715             y_r = [drone.position[1] for drone in reached_drones]
716             z_r = [drone.position[2] for drone in reached_drones]
717             self.ax_swarm.scatter(x_r, y_r, z_r, marker='*', c='g', s=100,
    label=f'Reached Goal ({len(reached_drones)})')
718
719         eliminated_drones = [d for d in self.swarm.drones if d.eliminated]
720
721
722
723
724
725
726
727         if self.swarm.mode == 'obstacle_avoidance':
728             for pillar in self.swarm.obstacles:
729                 theta = np.linspace(0, 2 * np.pi, 10)
730                 z_cyl = np.linspace(pillar.base_position[2], pillar.
    base_position[2] + pillar.height, 2)
731                 Z_mesh, THETA_mesh = np.meshgrid(z_cyl, theta)
732                 X_cyl = pillar.base_position[0] + pillar.radius * np.cos(
    THETA_mesh)
733                 Y_cyl = pillar.base_position[1] + pillar.radius * np.sin(
    THETA_mesh)
734                 self.ax_swarm.plot_surface(X_cyl, Y_cyl, Z_mesh, color='gray',
    alpha=0.6, rstride=1, cstride=1)
735
736
737             start_size = self.swarm.start_box_size
738             sb_center = self.swarm.start_box_center
739             x_s = [sb_center[0]-s/2 for s in [start_size, -start_size, -
    start_size, start_size, start_size]]
740             y_s = [sb_center[1]-s/2 for s in [start_size, start_size, -
    start_size, -start_size, start_size]]
741             z_s_bot = [sb_center[2]-start_size/2] * 5
742             z_s_top = [sb_center[2]+start_size/2] * 5
743             self.ax_swarm.plot(x_s, y_s, z_s_bot, color='green', linestyle='--'
    , label='Start Box')
744             self.ax_swarm.plot(x_s, y_s, z_s_top, color='green', linestyle='--'
    )
745             for i in range(4):
746                 self.ax_swarm.plot([x_s[i], x_s[i]], [y_s[i],y_s[i]], [z_s_bot[
    i], z_s_top[i]], color='green', linestyle='--')
747
748
749             end_size = self.swarm.end_box_size
750             eb_center = self.swarm.end_box_center
```

```
            x_e = [eb_center[0]-s/2 for s in [end_size, -end_size, -end_size, end_size, end_size]]
            y_e = [eb_center[1]-s/2 for s in [end_size, end_size, -end_size, -end_size, end_size]]
            z_e_bot = [eb_center[2]-end_size/2] * 5
            z_e_top = [eb_center[2]+end_size/2] * 5
            self.ax_swarm.plot(x_e, y_e, z_e_bot, color='red', linestyle='--', label='End Box')
            self.ax_swarm.plot(x_e, y_e, z_e_top, color='red', linestyle='--')
            for i in range(4):
                self.ax_swarm.plot([x_e[i], x_e[i]], [y_e[i],y_e[i]], [z_e_bot[i], z_e_top[i]], color='red', linestyle='--')


        plot_range = self.swarm.area_size / 2
        self.ax_swarm.set_xlim([-plot_range, plot_range])
        self.ax_swarm.set_ylim([-plot_range, plot_range])
        max_z_limit = max(self.swarm.area_size / 4 + 15/2 + 5, self.swarm.area_size / 2 + 5)
        self.ax_swarm.set_zlim([0, max_z_limit])
        self.ax_swarm.set_xlabel('X')
        self.ax_swarm.set_ylabel('Y')
        self.ax_swarm.set_zlabel('Z')
        self.ax_swarm.set_title(f"Drone Swarm Simulation ({self.swarm.mode.replace('_', ' ').title()} Mode)")
        if self.ax_swarm.get_legend_handles_labels()[1]:
            self.ax_swarm.legend()

        self.canvas_swarm.draw()

    def simulation_loop(self):
        """The main loop for running the simulation steps and updating the plot."""
        base_dt = 0.05

        while self.running:
            start_time_loop = time.time()
            dt = base_dt * self.swarm.simulation_speed

            self.swarm.update_swarm(dt)

            if self.swarm.drones or any(d.reached_goal for d in self.swarm.drones):
                self.master.after(1, self.plot_swarm)

            end_time_loop = time.time()
            elapsed_frame_time = end_time_loop - start_time_loop
            sleep_time = base_dt - elapsed_frame_time
            if sleep_time > 0:
                time.sleep(sleep_time / self.swarm.simulation_speed)
            elif sleep_time < 0:
                time.sleep(0.001)


    def start_simulation(self):
        """Starts or restarts the simulation."""
        if not self.running:
            self.running = True
            self.start_button.config(state=tk.DISABLED)
            self.stop_button.config(state=tk.NORMAL)

            self.swarm.reset_swarm()
            if self.swarm.drones:
```

```
                self.kp_var.set(self.swarm.drones[0].Kp_pid)
                self.ki_var.set(self.swarm.drones[0].Ki_pid)
                self.kd_var.set(self.swarm.drones[0].Kd_pid)
            self.plot_swarm()

            if self.simulation_thread is None or not self.simulation_thread.
    is_alive():
                self.simulation_thread = threading.Thread(target=self.
    simulation_loop)
                self.simulation_thread.daemon = True
                self.simulation_thread.start()

            if self.swarm.mode == 'obstacle_avoidance':
                self.info_label.config(text=f"Obstacle Avoidance Mode\nRunning
    ...")
            else:
                self.info_label.config(text=f"Pattern Formation Mode\nRunning
    ...")

    def end_simulation_run(self):
        """Called by Swarm when an obstacle avoidance simulation run finishes.
    """
        if self.swarm.mode == 'obstacle_avoidance':
            num_successful, num_eliminated, success_percentage, average_time =
    self.swarm.calculate_run_results()

            self.info_label.config(text=f"Obstacle Avoidance Mode\nRun Finished
    :\n"
                                        f"Successful: {num_successful}/{self.
    swarm.num_drones} ({success_percentage:.2f}%)\n"
                                        f"Eliminated (approx): {num_eliminated
    }\n"
                                        f"Avg Time (Successful): {average_time
    :.2f} s")


        self.running = False
        self.start_button.config(state=tk.NORMAL)
        self.stop_button.config(state=tk.DISABLED)
        print("Simulation run finished (or ended).")


    def stop_simulation(self):
        """Stops the currently running simulation."""
        if self.running:
            self.running = False


            self.start_button.config(state=tk.NORMAL)
            self.stop_button.config(state=tk.DISABLED)
            print("Simulation stopped manually.")
            if self.swarm.mode == 'obstacle_avoidance':
                self.info_label.config(text="Obstacle Avoidance Mode\
    nSimulation stopped.")
            else:
                self.info_label.config(text="Pattern Formation Mode\nSimulation
     stopped.")

    def update_pid_gains(self, event=None):
        """Updates PID gains for all drones."""
        try:
            new_kp = self.kp_var.get()
            new_ki = self.ki_var.get()
```

```
            new_kd = self.kd_var.get()
            if new_kp >= 0 and new_ki >= 0 and new_kd >= 0:
                for drone in self.swarm.drones:
                    drone.Kp_pid = new_kp
                    drone.Ki_pid = new_ki
                    drone.Kd_pid = new_kd
                print(f"Updated PID gains: Kp={new_kp}, Ki={new_ki}, Kd={new_kd}")
        except ValueError:
            print("Invalid PID gain value")


if __name__ == "__main__":
    root = tk.Tk()
    gui = SimulatorGUI(root)

    def on_closing():
        if gui.running:
            gui.stop_simulation()
        if gui.simulation_thread and gui.simulation_thread.is_alive():
            try:

                gui.simulation_thread.join(timeout=0.1)
            except Exception as e:
                print(f"Error joining simulation thread: {e}")
        root.quit()

    root.protocol("WM_DELETE_WINDOW", on_closing)
    root.mainloop()
```